ME 305 Mechatronics

Department of Mechanical Engineering

Cal Poly

Lab 5 Report: Closed and Open Loop Motor Control

Written by Gabe Choi and Maddie Howard

12/4/25

Introduction:

For our last lab, we explored closed and open loop motor control. The objectives of the lab were to design code to run a motor where the user could input new Vref, Kp, Ki values while also choosing if the system was running closed or open loop within the feedback loop. Most of the motor control code portions of the lab were given to us but the interrupt code needed for the feedback loop as well as updating the LCD screen in real time was designed by us. Once the code was completed, we completed a series of experiments with our motor, including steady-state and step response tests to find the natural frequency and damping ratio of our motor. We also used Simulink and MATLAB to simulate our system and do comparisons from our experimental setup to the simulation.

To start the lab, we needed to understand the full block diagram of our system, noting where the real-world hardware versus the software was located. Figure 1 showcases the block diagram for the closed and open loop motor control.

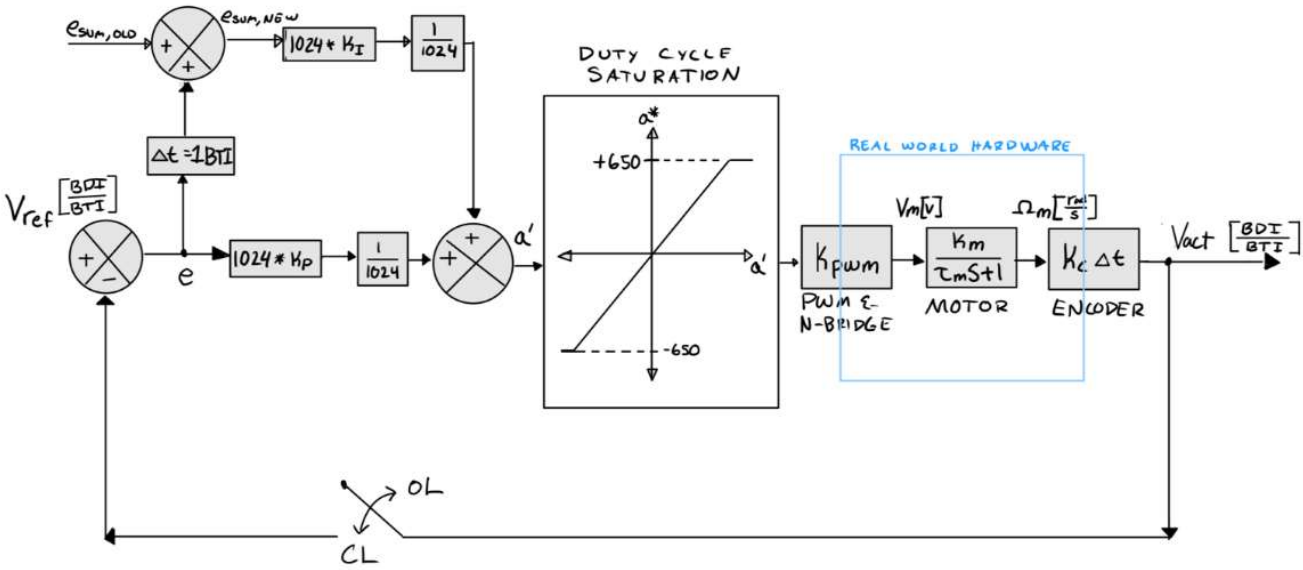


Figure 1. Block diagram of open and closed loop motor control with blue annotations for the real-world hardware in our system.

The equations shown in the block diagram are derived in detail in the section below. The rest of the report highlights how our code comes from the block diagram and our steady-state, step response and controller design experiments and results.

System Derivations:

1. Motor Transformer – 1st-Order and 2nd-Order Models

We start from the DC motor electrical and mechanical equations, assume armature inductance is negligible, and derive a 1st-order motor model. Then we add PI control, which gives a 2nd-order closed-loop system.

Electrical equation (inductance neglected):

$$V_{a(t)} = Ri + K_b\omega$$

Mechanical equation:

$$J\frac{d\omega}{dt} + b\omega = K_t i$$

Solve the electrical equation for current and substitute into the mechanical equation:

$$i = \frac{V_a - K_b\omega}{R}$$

$$J\frac{d\omega}{dt} + b\,\omega = \left(\frac{K_t}{R}\right)V_a - \left(\frac{K_tK_b}{R}\right)\omega$$

Group ω terms and define effective gain K_m and motor time constant τ_m:

$$J\frac{d\omega}{dt} = \left(\frac{K_t}{R}\right)V_a - \left(b + \frac{K_tK_b}{R}\right)\omega$$

$$K_m = \frac{K_t}{R\left(b + \frac{K_tK_b}{R}\right)}$$

$$\tau_m = \frac{J}{b + \frac{K_tK_b}{R}}$$

This gives the 1st-order motor differential equation:

$$\frac{\tau_m(d\omega)}{dt} + \omega = K_mV_a$$

Taking the Laplace transform and solving for $\frac{\Omega(s)}{V_{a(s)}}$ gives the 1st-order motor transformer:

$$G_{m(s)} = \frac{\Omega(s)}{V_{a(s)}} = \frac{K_m}{\tau_m s + 1}$$

In the lab, we include PWM, motor, encoder, and sampling effects in a single lumped gain $\mathbb{K}$ seen by the controller.

$$\mathbb{K} = K_{pwm}K_mK_c\Delta t$$

$$P(s) = \frac{V_{act(s)}}{U(s)} = \frac{\mathbb{K}}{\tau_m s + 1}$$

Here U(s) is the controller output (duty-cycle command).

The PI controller in continuous-time form is:

$$C(s) = \frac{K_p s + K_i}{s}$$

The open-loop transfer function is:

$$L(s) = C(s)P(s) = \mathbb{K}\frac{K_p s + K_i}{s(\tau_m s + 1)}$$

With unity feedback, the closed-loop motor transformer from reference V_ref(s) to measured velocity V_act(s) is:

$$\frac{V_{act(s)}}{V_{ref(s)}} = \mathbb{K}\frac{K_p s + K_i}{\tau_m s^2 + (1 + \mathbb{K}K_p)s + \mathbb{K}K_i}$$

This is a 2nd-order system. Setting $K_i = 0$ (proportional-only control) reduces the closed-loop transfer function to:

$$\frac{V_{act(s)}}{V_{ref(s)}} = \frac{\mathbb{K}K_p}{\tau_m s + (1 + \mathbb{K}K_p)}$$

This has the standard 1st-order form with closed-loop time constant τ_{cl} and DC $Gain_{cl}$:

$$Gain_{cl} = \frac{\mathbb{K}K_p}{1 + \mathbb{K}K_p}$$

$$\tau_{cl} = \frac{\tau_m}{1 + \mathbb{K}K_p}$$

2. Open-Loop vs Closed-Loop K_m and Time Constant

The physical motor gain $K_m$and time constant τ_m are properties of the motor and load and do not change when we close the loop. However, the way we estimate them from data depends on whether the system is open-loop or closed-loop.

Open-loop motor transformer from applied voltage to speed:

$$G_{m(s)} = \frac{K_m}{\tau_m s + 1}$$

In the open-loop experiments, the slope of the steady-state $V_{ref} - V_{act}$ plot gives an estimate of the lumped gain $K_{open} = K_p K_{pwm} K_m K_c$ (for proportional control). From this, we solve for K_m:

$$K_{open} = Slope_{open}$$

$$K_{m(open)} = \frac{K_{open}}{K_p K_{pwm} K_c}$$

The open-loop time constant τ_m is found directly from the open-loop step response of $V_{act}(t)$, for example using the 63.2% rise time of a 1st-order system.

When the loop is closed with proportional control, the closed-loop transfer function becomes 1st-order with:

$$\frac{V_{act(s)}}{V_{ref(s)}} = \frac{\mathbb{K}\,K_p}{\tau_m s + \left(1 + \mathbb{K}\,K_p\right)}$$

The closed-loop time constant τ_{cl} is shorter than τ_m by a factor (1 + $\mathbb{K}$ K_p):

$$\tau_{cl} = \frac{\tau_m}{1 + \mathbb{K}\,K_p}$$

If we try to back-solve for an apparent motor gain from closed-loop data using the 1st-order fit, the slope relates to the combination ($\mathbb{K}$ K_p)/(1 + $\mathbb{K}$ K_p). This can make the numerical value of $K_{m(closed)}$ look different from $K_{m(open)}$, even though the underlying motor K_m is the same. The difference comes from the closed-loop dynamics, not a real change in the motor.

3. K_{pwm} and $K_c(\Delta t)$– Lumped Gain Derivation

The lumped gain $\mathbb{K}$ represents how a change in controller output (duty-cycle command) shows up as a change in the measured velocity V_{act}. It combines PWM scaling, motor gain, encoder scaling, and sampling.

We define:

$$\mathbb{K} = K_{pwm}K_mK_c\Delta t$$

Each term has the following meaning:

K_{pwm}: maps digital duty-cycle units from the controller to average motor voltage V_a.

K_m: continuous-time motor gain from V_a.to angular velocity ω $\left(\frac{rad}{s}\right)$.

K_c: encoder gain from angular velocity ω to measured velocity counts per second.

Δt: sampling period (seconds per control update).

Putting these together: a unit step at the controller output produces K_{pwm} volts at the motor, which produces K_mK_{pwm} $\left(\frac{rad}{s}\right)$ at steady state, which the encoder converts to $K_cK_mK_{pwm}$ counts per second, and multiplying by Δt gives $\mathbb{K}$ counts per sample at the measured output V_{act}.

4. Damping Ratio and Natural (Nominal) Frequency

With PI control, the closed-loop transfer function is 2nd order. We can express its denominator in the standard 2nd-order form to identify the damping ratio ζ and natural frequency ω_n, or we can estimate them from the measured step response.

Closed-loop transfer function denominator (from Section 1):

$$\tau_m s^2 + \left(1 + \mathbb{K}\,K_p\right)s + \mathbb{K}\,K_i = 0$$

Divide through by τ_m to match the standard 2nd-order form $s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$:

$$s^2 + \left(\frac{1 + \mathbb{K} K_p}{\tau_m}\right) s + \frac{\mathbb{K} K_i}{\tau_m} = 0$$

By comparing coefficients with $s^2 + 2\zeta\omega_n s + \omega_n^2$, we identify ω_n and ζ in terms of controller gains and motor parameters:

$$\omega_n^2 = \frac{\mathbb{K} K_i}{\tau_m}$$

$$2\zeta\omega_n = \frac{1 + \mathbb{K} K_p}{\tau_m}$$

Therefore:

$$\omega_n = \text{sqrt}((\mathbb{K} K_i)/\tau_m$$

$$\zeta = \left(\frac{1 + \mathbb{K} K_p}{2\tau_m\omega_n}\right)$$

Interrupt Description:

Using the block diagram in Figure 1 above, we were able to step by step design and implement our interrupt code. We first needed to read the encoder within the motor and store the position value and velocity of the motor and find the change in position. Since the user can choose if the motor is running on an open or closed loop we had to then check and set the control mode designated. Using algebraic equations, we were able to find the error by subtracting the actual velocity from the reference velocity and display that variable on the screen. Using the block diagram expressions, we completed the full PI control calculation and saturating addition to finding the new velocity for the motor. We also found the effort of the motor and had to add DAC voltage scale so that we could see the output velocity on the oscilloscope from DAC A on our kit. With all our values updated and the complete block diagram in our code, we were able to run the motor with both closed and open loop control while showcasing the Vref, Vact, Error and Effort on the LCD screen. Our full code can be found at the end of our report.

Steady State Tests:

Our first experiment with our motor setup was steady state tests to find our Km value for our motor. We started by running open loop tests and setting Vref from values of 10 to 160 while collecting the Vact value on our LCD screen. We completed this 4 times with different Kp values. Figure 2 below shows the Vref plotted against the Vact for the 4 different Kp values tested. The slopes for each of the lines are highlighted on the right side of the graph. We also used the constant parameters known from our lab as explained in section 3 of the derivations above.

Table 1. Constant Parameters used to find Km in both Closed and Open Loop Operation

Constant Parameters	
Kpwm	KcΔt
0.0369	0.6370

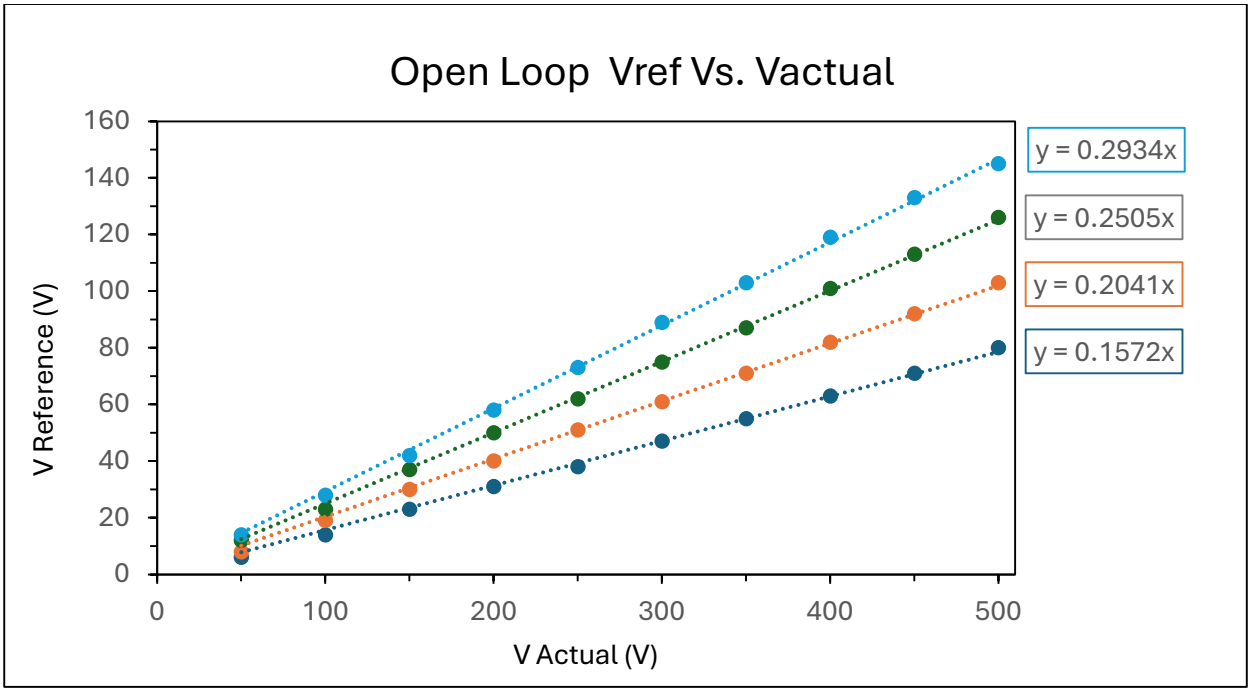


Figure 2. Vref versus Vact for 4 different Kp values in open loop operation. Kp = 0.7 is in dark blue, Kp = 0.9 is in orange, Kp = 1.1 is in green and Kp = 1.3 is in blue.

From the equations discussed above, we know that the slope of the lines in this graph is equal to $\mathbb{K}\,K_p$ and knowing that $\mathbb{K} = K_{pwm}K_mK_c\Delta t$ we were able to find the average Km value for our motor with open loop operation. Table 2 has the values found from our open loop steady-state experiment with our motor.

Table 2. Open Loop Steady-State Experiment Values to find Km

Open Loop				
Kp	Slope (KpKapk)	KapK	KapKavg	Km
0.700	0.157	0.225	0.226	9.623
0.900	0.204	0.227		
1.100	0.251	0.228		
1.300	0.293	0.226		

After completing the open loop experiment, we followed the exact same steps for the closed loop steady-state test. Figure 3 showcases the plot of Vref versus Vact for the different Kp values and Table 3 has the values found from the closed loop tests.

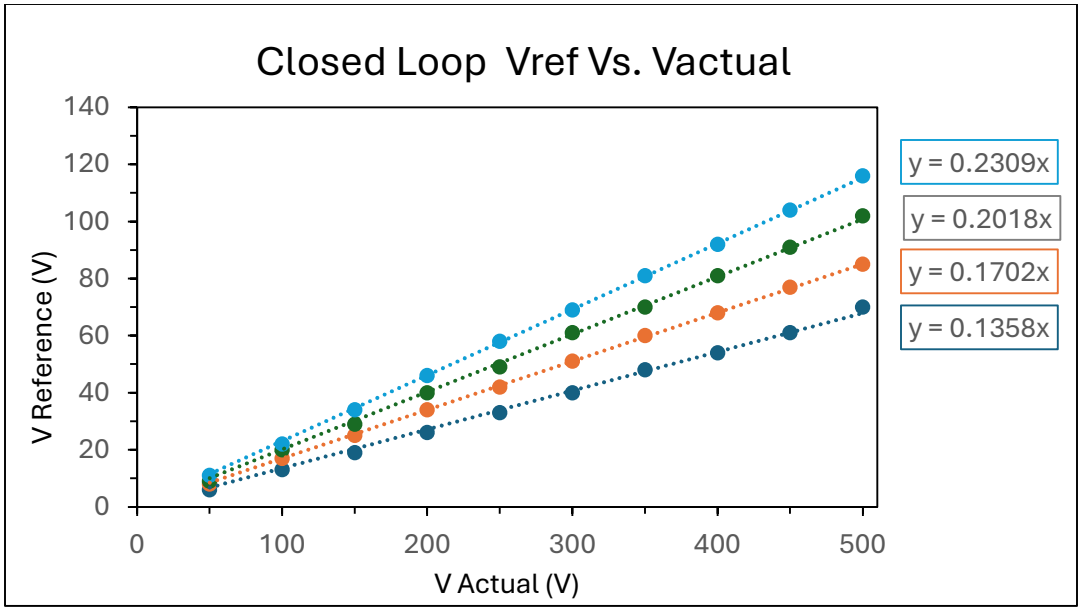


Figure 3. Vref versus Vact for 4 different Kp values in closed loop operation. Kp = 0.7 is in dark blue, Kp = 0.9 is in orange, Kp = 1.1 is in green and Kp = 1.3 is in blue.

Table 3. Open Loop Steady-State Experiment Values to find Km

Closed Loop				
Slope	KpKapK	KapK	KapKavg	Km
0.136	0.120	0.171	0.151	6.443
0.170	0.145	0.162		
0.202	0.168	0.129		
0.231	0.188	0.144		

The closed loop steady-state results highlight how the feedback control changes the apparent gain of the motor. In the closed loop application, the controller modifies the input effort to force the output to change, which introduces the new $1 + \mathbb{K}\,K_p$ term. As a result, the slopes in the closed loop graph are lower than the slopes in the open loop graph even with the same Kp values.

Step Response Tests:

The next tests we ran were two step response tests to find the time constant for both the open and closed loop operation. For this, we had to set up the oscilloscope to capture the response right when we turned on the motor. For the open loop run, the time constant is equal to 63.2% of the steady-state value. Using the scope image from the open loop run we were able to determine the time constant to be 10.40ms. For the closed loop run, the time it takes to reach 63.2% of the steady-state value isn't the time constant of the motor but is instead shorter than the actual time constant, as explained in the derivation section of our report. Using the equation and time constant given on the oscilloscope of 12.20ms we found the motor time constant for the closed loop control to be 41.31ms. Figure 4 below shows both annotated scope images from the open and closed loop control step response tests.

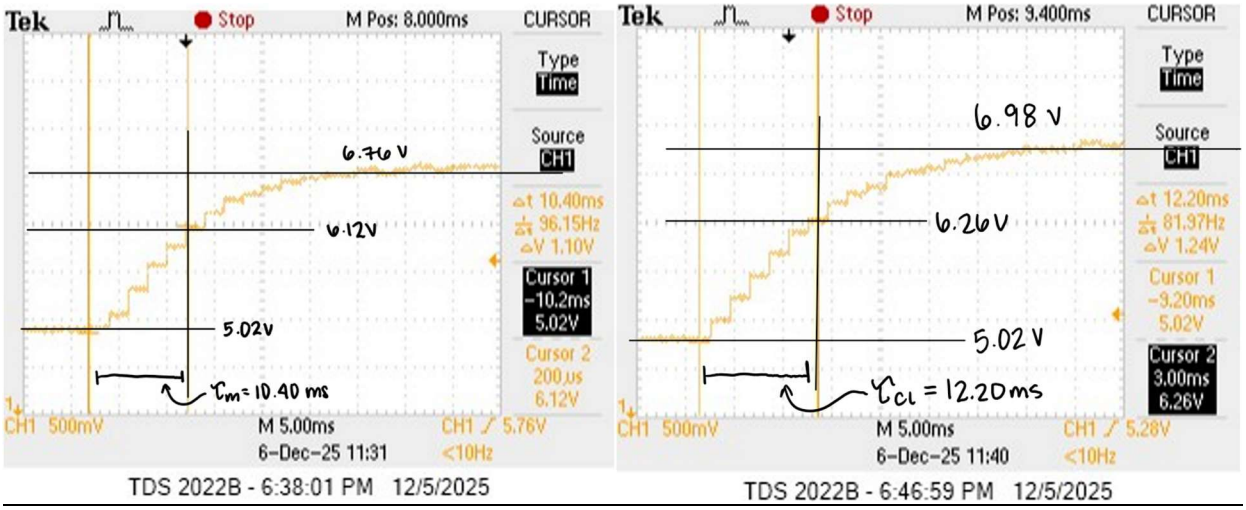


Figure 4. Oscilloscope images for step-response tests annotated with time constant and important voltage readings. Open loop control on the right and closed loop control on the left.

Using the time constants found we were able to find the natural frequency and damping ratio. Table 4 shows the values found for both open and closed loop control.

	Open Loop	Closed Loop
ω_n (rad/s)	34.5	49.5
ζ	1.12	1.12

Table 4. Open loop and closed loop natural frequency and damping ratio from time constants determined in step-response tests.

The damping ratios as shown in the table, show that the motor is highly damped in both the open and closed loop cases, because the damping ratio, Zeta, is greater than one. This agrees with the scope image, as the system never overshoots the steady state value. These values were significant in our controller design, because they provide the foundation to ensure that any new gains used meets the desired performance without causing instability or overshooting the steady state value.

Controller Design:

Once we had completed our steady-state and step-response tests, we were able to determine a new set of gains, Kp* and Ki*, that made the closed-loop system have a new natural frequency 1.6 times larger than the nominal frequency found above and a damping ratio equal to one. With our numbers above, that means that the new natural frequency for the closed-loop system is 55.2 rad/s and the damping ratio is 1. Using the same equation but in reverse, you can solve for the new gains, Kp* and Ki*, which are equivalent to Kp* = 3.83 and Ki* = 0.576 for our motor.

Using a pre-made Simulink file, we modified the file to have our parameters, Kp*, Ki*, Km and τ_m, with a Vref of 100. We ran the simulation and plotted the results in MATLAB. We also carried the same experiment using our kit and motor, with the new gain values and the Vref of 100. After obtaining the oscilloscope reading for the new parameters, we were able to also add that to MATLAB and plot the simulation and experimental results on the same plot. Figure 5 below showcases the simulation and experimental results for the new gains overlayed on the same plot.

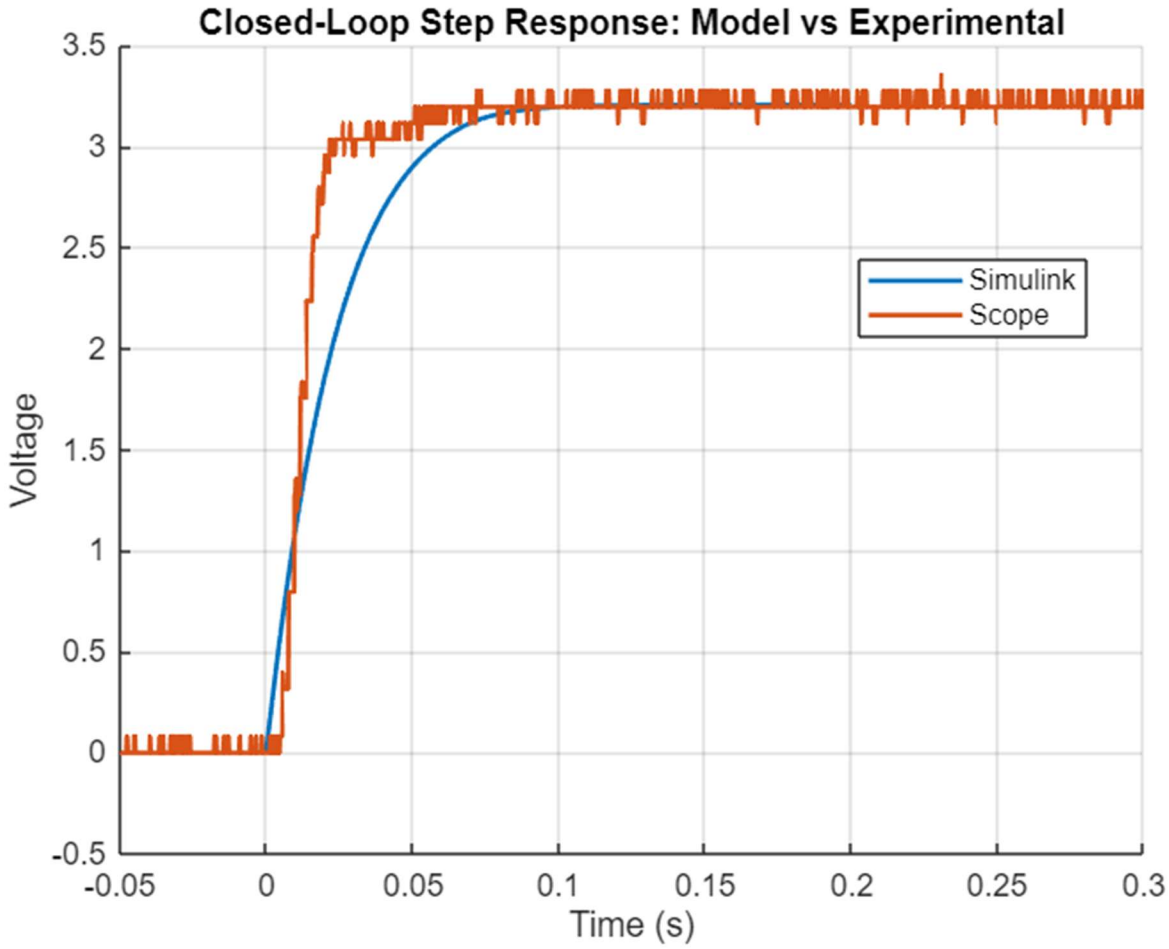


Figure 5. Closed Loop Step Response for both model and experiment using new gains. Simulation curve is shown in blue and the experimental curve is shown in red.

From the overlayed plot we can see that our experiment does follow the same curve as the simulation, but it is important to note that the experiment increases slightly slower and decreases slightly slower. This can be seen as the red curve starts slightly after the blue curve and has a larger gap as it gets closer to the steady-state value. This is because the real motor must build up the inductance as it starts to spin, causing it to "lag" before starting to increase in speed. The simulation doesn't show the inductance build up, which is why it looks like it starts earlier.

This plot when compared to the step-response plots above, also shows that with the new gains, the controller increased the speed of the system while maintaining a critically damped response. Overall, we were able to produce predictable and repeatable improvements in closed-loop motor behavior and experiments validated the modeling process and control design process. We were able to identify the motor dynamics, modify parameters and solve new gains to produce a redesigned controller.

```
;*******************************************************************************
;* Lab 5 main_ISR                                                              *
;*******************************************************************************
;* Summary:                                                                    *
;*  This code is to be used with the 2016 hardware for ME305. This code is to be an   *
;* interrupt service routine that implements a proportional plus integral controller  *
;* on the velocity of the ME 305 dc motor with encoder system.  A full user interface *
;* is provided by the ME305LibV3.2.lib.                                        *
;*                                                                             *
;* Author: William R. Murray, Eric Espinoza-Wade, and Charlie T. Refvem        *
;*  Cal Poly University                                                        *
;*  Fall 2024                                                                  *
;*                                                                             *
;*******************************************************************************

;/-----------------------------------------------------------------------------\
;| Include all associated files                                                |
;\-----------------------------------------------------------------------------/
; The following are external files to be included during assembly


;/-----------------------------------------------------------------------------\
;| External Definitions                                                        |
;\-----------------------------------------------------------------------------/
; All labels that are referenced by the linker need an external definition

              XDEF  main

              XDEF  RUN, CL, V_ref, Theta_OLD, KP, KI, UPDATE_FLG1
;/---------------------- ------------------------------------------------------\
;| External References                                                         |
;\-----------------------------------------------------------------------------/
; All labels from other files must have an external reference

              XREF  ENABLE_MOTOR, STARTUP_MOTOR, UPDATE_MOTOR, DISABLE_MOTOR
              XREF  STARTUP_PWM, STARTUP_ENCODER, READ_ENCODER, DISABLE_ENCODER
              XREF  OUTDACA, OUTDACB
              XREF  INITLCD, CLRSCREEN, SETADDR, GETADDR, CURSOR_ON, DISP_OFF
              XREF  OUTCHAR, OUTCHAR_AT, OUTSTRING, OUTSTRING_AT
              XREF  INITKEY, LKEY_FLG, GETCHAR
              XREF  Entry, ISR_KEYPAD

              XREF  V_act_DISP, ERR_DISP, EFF_DISP, INTERVAL, IFENTRY

;/-----------------------------------------------------------------------------\
;| Assembler Equates
;\-----------------------------------------------------------------------------/
; Hardware register addresses
TFLG1         EQU $004E             ; Timer interrupt flag register 1
TC0           EQU $0050             ; Timer channel 0 register
PORTT         EQU $0240             ; Port T data register

; Bit masks
C0F           EQU %00000001         ; Timer channel 0 flag bit

; Control limits
LOWER_LIM     EQU -650              ; Maximum reverse duty cycle
UPPER_LIM     EQU 650               ; Maximum forward duty cycle

; Saturation values
Max_Pos       EQU $7FFF             ; Maximum positive saturated value
```

```
Max_Neg       EQU $8000             ; Minimum saturated value (most negative)

;/-------------------------------------------------------------------------------\
;| Variables in RAM
;\-------------------------------------------------------------------------------/
DEFAULT_RAM:  SECTION

; System state flags
RUN:          DS.B 1                ; Controller enable flag (0=stopped, 1=running)
CL:           DS.B 1                ; Control mode (0=open-loop, 1=closed-loop)

; Velocity variables
V_ref:        DS.W 1                ; Reference (desired) velocity
V_act:        DS.W 1                ; Actual (measured) velocity
V_End:        DS.W 1                ; Final velocity output value

; Encoder position tracking
Theta_NEW:    DS.W 1                ; Current encoder position
Theta_OLD:    DS.W 1                ; Previous encoder position

; Controller gains (scaled by 1024)
KP:           DS.W 1                ; Proportional gain
KI:           DS.W 1                ; Integral gain

; PI term calculations
AP:           DS.W 1                ; Proportional term after gain scaling
AI:           DS.W 1                ; Integral term after gain scaling
A_Prime:      DS.W 1                ; Total control output (AP + AI)

; Control calculations
ERR:          DS.W 1                ; Velocity error (V_ref - V_act)
EFF:          DS.W 1                ; Motor effort as percentage
ESUM:         DS.W 1                ; Accumulated error (integral term)

; Motor actuation
MOTOR_POWER:  DS.W 1                ; Total motor command (saturated)
P_POWER:      DS.W 1                ; Proportional term contribution
I_POWER:      DS.W 1                ; Integral term contribution

; Display update control
UPDATE_COUNT: DS.B 1                ; Counter for display refresh (decrements each ISR)
UPDATE_FLG1:  DS.B 1                ; Display update flag for line 1

; Encoder storage
ENCODER_COUNT: DS.W 1               ; Raw encoder count from READ_ENCODER

;/-------------------------------------------------------------------------------\
;| Main Program Code
;\-------------------------------------------------------------------------------/
MyCode:       SECTION

main:
              jsr IFENTRY           ; Initialize Lab 5 interface
spin:
              bra spin              ; Endless loop - all work done in ISR

;/-------------------------------------------------------------------------------\
;| Interrupt Service Routine - Timer Channel 0
;\-------------------------------------------------------------------------------/
;* Summary:
;*   Main control loop ISR executing at regular intervals defined by INTERVAL.
```

```
;*   Performs velocity measurement, PI control calculation, and motor actuation.
;*
;* Algorithm:
;*   1. Measure velocity from encoder change
;*   2. Calculate error (V_ref - V_act)
;*   3. Update integral term (ESUM)
;*   4. Compute PI control: u = (KP*ERR + KI*ESUM)/1024
;*   5. Saturate output to  650
;*   6. Update motor and display
;*******************************************************************************

TC0ISR:
              bset PORTT, $80        ; Set PT7 high (ISR timing measurement)

;--- Velocity Measurement ---
              movw Theta_NEW, Theta_OLD ; Shift current to previous
              jsr READ_ENCODER       ; Read new encoder position -> D
              std Theta_NEW          ; Store new position
              subd Theta_OLD         ; Calculate change: ?Theta = new - old
              std V_act              ; Velocity = ?Theta / ?t (fixed ?t)

;--- Control Mode Selection ---
              tst CL                 ; Check control mode
              bne CLOSED             ; Branch if closed-loop

;--- Open-Loop Mode ---
OPEN:
              movw V_ref, ERR        ; In open-loop, error = reference
              bra ASTAR              ; Skip integral update

;--- Closed-Loop Mode ---
CLOSED:
              ldd V_ref              ; Load reference velocity
              subd V_act             ; Calculate error: e = V_ref - V_act
              std ERR                ; Store error

              ; Update integral term: ESUM += ERR (with saturation)
              ldy ESUM               ; Load current integral sum
              jsr SDBA               ; Saturated add: ESUM + ERR
              std ESUM               ; Store updated integral sum

;--- PI Control Calculation ---
ASTAR:
              ; Integral term: AI = (KI * ESUM) / 1024
              ldd ESUM
              ldy KI
              emuls                  ; Extended multiply: Y:D = ESUM * KI
              ldx #1024
              edivs                  ; Extended divide: Y = quotient, D = remainder
              sty AI                 ; Store integral contribution

              ; Proportional term: AP = (KP * ERR) / 1024
              ldd ERR
              ldy KP
              emuls                  ; Extended multiply: Y:D = ERR * KP
              ldx #1024
              edivs                  ; Extended divide: Y = quotient, D = remainder
              sty AP                 ; Store proportional contribution

              ; Total control: A_Prime = AP + AI
              ldd AI
```

```
              jsr SDBA               ; Saturated add with AP in Y

              ; Saturate to actuator limits
              cpd #UPPER_LIM         ; Compare with upper limit
              bge LARGE              ; Branch if result >= 650
              cpd #LOWER_LIM         ; Compare with lower limit
              ble SMALL              ; Branch if result <= -650
              std A_Prime            ; Within limits, store result
              bra EFFCALC

LARGE:
              movw #UPPER_LIM, A_Prime ; Saturate to +650
              bra EFFCALC

SMALL:
              movw #LOWER_LIM, A_Prime ; Saturate to -650

;--- Effort Calculation ---
EFFCALC:
              ; Calculate percentage effort: EFF = (A_Prime * 100) / 650
              ldd A_Prime
              ldy #100
              emul                   ; Y:D = A_Prime * 100
              ldx #650
              idiv                   ; X = quotient (percentage)
              stx EFF                ; Store effort percentage

              ; Output velocity to DAC A for oscilloscope monitoring
              ldd V_act
              ldy #13
              emuls                  ; Scale velocity
              addd #2048             ; Add offset for DAC
              jsr OUTDACA            ; Output to DAC A

;--- Display Update Management ---
INTERRUPTS:
              dec UPDATE_COUNT       ; Decrement display counter
              tst UPDATE_COUNT       ; Check if zero
              beq UPLINE1            ; Update display every 256 interrupts
              bra RUNCHK

UPLINE1:
              ; Copy current values to display variables
              movw ERR, ERR_DISP
              movw EFF, EFF_DISP
              movw V_act, V_act_DISP
              movb #$01, UPDATE_FLG1 ; Set display update flag

;--- RUN State Management ---
RUNCHK:
              tst RUN                ; Check if controller is running
              bne MOTOR              ; If running, update motor

              ; Controller stopped - reset integrator and position
              clr ESUM               ; Clear accumulated error
              jsr READ_ENCODER       ; Read current position
              std Theta_OLD          ; Store as reference for restart
              movw #$8000, A_Prime   ; Set motor to neutral position

;--- Motor Actuation ---
MOTOR:
```

```
              ldd A_Prime            ; Load motor command
              jsr UPDATE_MOTOR       ; Send to motor driver
              bra NOT_YET

;--- ISR Cleanup ---
NOT_YET:
              ; Schedule next interrupt
              ldd TC0                ; Get current timer value
              addd INTERVAL          ; Add interval for next interrupt
              std TC0                ; Set next compare value
              bset TFLG1, C0F        ; Clear channel 0 interrupt flag

              bclr PORTT, $80        ; Clear PT7 (end ISR timing)
              rti                    ; Return from interrupt

;/-------------------------------------------------------------------------------\
;| Subroutine: SDBA - Saturated Double Byte Adder
;\-------------------------------------------------------------------------------/
;* Summary:
;*   Adds D register to Y register with overflow saturation.
;*
;* Input:
;*   D = first operand
;*   Y = second operand
;*
;* Output:
;*   D = saturated result (D + Y)
;*   Y = preserved
;*
;* Algorithm:
;*   If (D + Y) overflows, saturate to $7FFF (positive) or $8000 (negative)
;*   based on the sign of the operands.
;*******************************************************************************

SDBA:
              pshy                   ; Save Y on stack
              addd 0, SP             ; Add: D = D + Y
              bvc DONE               ; If no overflow, result is valid

              ; Overflow occurred - determine saturation direction
              tst 0, SP              ; Check sign of Y (operand)
              bmi NEG                ; If negative, saturate to minimum

              ldd #Max_Pos           ; Saturate to maximum positive
              bra DONE

NEG:
              ldd #Max_Neg           ; Saturate to maximum negative

DONE:
              ins                    ; Clean up stack
              ins
              rts                    ; Return with result in D

;/-------------------------------------------------------------------------------\
;| Interrupt Vectors
;\-------------------------------------------------------------------------------/
              ORG $FFFE              ; Reset vector
              DC.W Entry

              ORG $FFCE              ; Keypad interrupt vector
```

```
DC.W ISR_KEYPAD

ORG $FFEE               ; Timer channel 0 interrupt vector
DC.W TC0ISR
```